Codebase Change Summary — Week 9

Integration Testing

SE 413/513 — Software Testing

Demo Testing Suite

What changed in `demo-testing-suite/` since Week 8, and why

1 Purpose of This Document

The `demo-testing-suite/` codebase is a single, shared project that grows one lecture at a time rather than being rebuilt from scratch each week. This document summarizes exactly what was added or changed for Week 9 and, more importantly, explains *why* each change exists — which concept from the Week 9 reading, *Integration Testing: Testing How Components Work Together*, it demonstrates. Read this alongside the reading and `demo-testing-suite/README.md`.

Key Idea

Nothing from Weeks 1–8 was removed. This week does two things. It adds a self-contained project that runs every example in the reading: a small shop with an HTTP API, a database, and a (fake) third-party payment provider, its interface specifications, a written design review, and two deliberately broken components. And it applies integration testing to the *real* suite for the first time — which found two real problems there.

2 What Changed At a Glance

File	Status	Purpose
The main suite		
`tests/integration/conftest.py`	New	Assembles `UserRegistrationService` from real parts: the real `UserRepository`, a SQLite file, and a fake email client.
`tests/integration/test_registration.py`	New	Three passing integration tests, and one strict `xfail` documenting a defect they found.
`app/accounts/user_repository.py`	Modified	Adds `create_schema()`: the production `users` schema, which until now existed only in a test fixture.
`pytest.ini`	Modified	Registers the `integration` marker.
The reading's examples (not in the default run)		
`demos/integration_testing/shop/`	New	The shop: Flask API, order service, repository, SQLite schema, payment client.
`demos/integration_testing/shop/payments_v1.py`, `service_buggy.py`	New	Two broken components: a client that sends dollars instead of cents, and a service with a transaction bug.
`demos/integration_testing/fakes/`	New	A fake payment provider that enforces the provider's contract, served over real HTTP.
`demos/integration_testing/specs/`	New	Both API specifications, and the design/API review (findings R1–R11, each traced to its tests).
`demos/integration_testing/tests/`	New	46 tests: unit, bottom-up, top-down, stubbed, live, and fully assembled.
`demos/integration_testing/serve.py`, `README.md`	New	Run the shop for `curl`; every command with its expected output.
Tooling and documentation		
`requirements.txt`	Modified	Adds `flask`, `requests`, and `responses`.
`docs/DEMO_GUIDE_week09_integration_testing.md`	New	Section-by-section lecture companion for the reading.
`docs/DEMO_GUIDE_week04_tdd_test_doubles.md`	Modified	A forward note: the Week 4 registration service is now also integration-tested.
`README.md` (suite and repository)	Modified	Layout, commands, expected results, and the Weekly Updates index.

Tip

Install the new dependencies before running anything this week:

```
cd demo-testing-suite
pip install -r requirements.txt
```

3 Integration Testing Comes to the Main Suite

Locations: `tests/integration/`, `app/accounts/user_repository.py`

Since Week 4, `UserRegistrationService` has been tested with a *mocked* `UserRepository` (`tests/`

`accounts/test_registration_service.py`). Those tests are still there and still useful. The new folder tests the same service assembled from real parts, and every test in it is marked:

```
pytestmark = pytest.mark.integration
```

```
pytest -m integration          # 3 passed, 75 deselected, 1 xfailed
pytest -m "not integration"    # 1 failed, 72 passed, 4 deselected, 2 xfailed
```

The second line is exactly Week 8's result.

Writing these tests found two problems, both real:

1. **The schema lived only in a test.** The `users` table had been created by a test fixture since Week 2, and nowhere in `app/`. An integration test has to use the application's own schema, so `create_schema()` now exists. It also makes `name NOT NULL UNIQUE`, so the database rejects a duplicate even from code that bypasses the service.
2. **A half-registered user.** `register()` saves the user, *then* sends the welcome email. If the mail server is down, the user is saved but never welcomed — and cannot try again, because a second registration is rejected as a duplicate. A mocked repository has no saved state to leave behind, so no unit test could see it.

Common Pitfall

The second problem is documented with a strict `xfail`, not fixed — the same way Week 1's late-fee bug and Week 3's shipping bug were. Fixing it is Exercise 4 in the reading. When you do, `strict=True` makes the test *fail* (`XPASS(strict)`) until you remove the marker.

4 The Reading's Shop, Runnable

Location: `demos/integration_testing/`. **Run its commands from inside the folder:**

```
cd demos/integration_testing
pytest                         # 46 passed, in about a second
```

The shop has six integration points (IP1–IP6): client to API, API to service, service to repository, repository to SQLite, service to payment client, and payment client to the third-party provider over HTTP. Each test file makes a different set of them real:

Test file	What is real	Strategy step
`test_order_service_unit.py`	only the service (all mocked)	unit baseline
`test_repository.py`	repository + SQLite	bottom-up (driver)
`test_payments_stubbed.py`	client; provider stubbed (`responses`)	stub at HTTP
`test_payments_live.py`	client + fake provider, real HTTP	bottom-up
`test_order_service.py`	service + DB + fake provider	bottom-up, step 2
`test_api_topdown.py`	API; service stubbed	top-down
`test_api.py`	everything	assembled system

Two broken components, one switch each

Environment variables swap in the deliberately broken versions, for one command only. The *v1 client* sends `"12.50"` dollars instead of `1250` cents; the *buggy service* commits the stock and the order in two separate transactions:

```
SHOP_PAYMENTS=v1   pytest tests/test_api.py
SHOP_SERVICE=buggy pytest tests/test_order_service.py
```

Run	v1 client	buggy service	What it shows
`test_order_service_unit.py`	3 passed	3 passed	mocks cannot see either defect
`test_payments_stubbed.py`	1 failed	5 passed	the loose stub passes; the contract check fails
`test_payments_live.py`	3 failed	6 passed	the provider answers `400`: "integer number of cents"
`test_order_service.py`	2 failed	1 failed	v1: charge rejected; buggy: `assert 3 == 5`, stock leaked
`test_api.py`	2 failed	5 passed	`assert 502 == 201`: *where*?

Read the table by column. The dollars-for-cents defect is caught at every level from the contract check upward, but only the lower-level tests say *where* it is. The transaction defect is caught at exactly one level — the service against a real database — and the end-to-end API tests miss it, because the API's input validation blocks the input that triggers it.

The design review

`specs/design_review.md` records a review held before the tests were written: the integration points, the checklist, and eleven findings (R1–R11). Each finding names the tests that check it, and each of those tests names its finding in its docstring. That traceability is the pattern the reading asks you to follow in your semester project.

Trying the API by hand

```
python serve.py                    # shop on :8000, fake provider on :8001
curl -i -X POST http://127.0.0.1:8000/orders -H 'X-API-Key: shop-key' \
     -H 'Content-Type: application/json' \
     -d '{"sku": "MUG", "quantity": 2, "customer_email": "ana@example.com"}'
```

returns `201 CREATED` with a `Location` header. The folder's `README.md` lists the 400, 401, 409, 402, and 503 variations.

5 Updated: Expected Test Results

From the project root:

```
cd demo-testing-suite
pip install -r requirements.txt
pytest -v
```

you should now see **75 passed, 3 xfailed, 1 failed**: three more passing tests than Week 8, the same Week 1 and Week 3 non-passes as before, and the one new strict `xfail` described above. Every `app/` module is still at 100% branch coverage. `demos/integration_testing/` is not part of that run (`pytest.ini` sets `testpaths = tests`); run it from inside its folder.

6 How This Connects to the Reading

Key Idea

Weeks 2–8 tested components one at a time. This week tests the promises between them: the formats, transactions, credentials, and failure behavior that two components must agree on. Both demo defects pass every unit test, and both real defects in the main suite were invisible to the Week 4 mocks. Each was found by connecting the real parts, or — before that — by reviewing the design.

7 Suggested Next Steps

1. Run the switch table above yourself, one cell at a time, and *predict* each result before you run it.
2. Before reading `specs/design_review.md`, review `specs/payment_provider_api.md` yourself with the reading's checklist. How many of R1–R11 did you find?
3. Fix the half-registered user (Exercise 4). Decide between deleting the user when the email fails and saving it with a "welcome pending" flag. Remove the `xfail` marker when your fix lands.
4. Implement refunds in the demo (Exercise 3), writing the five IT-R test cases first.
5. Find out what `PaymentClient.charge()` does when the provider returns `201` without an `id` (Exercise 5), and decide what it should do.